SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 35%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I K.AISHWARYA(192572116) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Scenario Based Assignment

1. AI-Powered Drone Routing for Emergency Medicine Delivery

A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. Roads and landing areas may become inaccessible, weather can change travel costs, and the drone receives only partial real-time information. The main objective is to reach the destination quickly while controlling risk and using the available information effectively.

1.1 Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

Greedy Best-First Search (GBFS) selects the next location that appears closest to the destination according to a heuristic. In this scenario, the heuristic can be the straight-line distance from a possible drone position to the medicine delivery point. The algorithm uses h(n), where h(n) estimates the remaining distance or time, and expands the available node with the smallest heuristic value.

For example, suppose the drone can travel through locations A and B. If the estimated remaining distances are h(A) = 4 km and h(B) = 7 km, GBFS chooses A first because it appears closer to the destination. This can quickly direct the drone toward the target without spending much time evaluating every possible route.

The main strength of GBFS is speed. It is useful when emergency medicine must be delivered quickly and a reasonable route must be found without examining the entire search space. It can also work with a simple distance estimate and does not require a detailed cost calculation for every route.

However, GBFS does not fully consider the cost already spent to reach a location. A route that looks close may contain strong winds, dangerous landing areas, or blocked passages. For example, a route with 2 km remaining may require 15 minutes because of severe wind, while another route with 4 km remaining may be safe and take only 5 minutes. GBFS may choose the first route because it focuses on apparent closeness.

Under uncertainty, its decision can become outdated when a path is blocked or weather changes. Therefore, GBFS is fast but does not guarantee the shortest, safest, or lowest-cost route. It should be combined with safety checks and frequent updates.

1.2 Explain how A* would handle the same situation, including how it balances cost and heuristic

A* Search evaluates both the cost already incurred and the estimated cost remaining. Its evaluation function is:

$f(n) = g(n) + h(n)$

Where:

- g(n): The actual cost from the starting point to the current node.
- h(n): The estimated cost from the current node to the destination.
- f(n): The estimated total cost of the route through the current node.

For a drone, the cost can represent travel time, energy use, weather exposure, and a penalty for unsafe areas.

Suppose route A has already cost 8 minutes and its estimated remaining time is 4 minutes.

f(A) = 8 + 4 = 12 minutes

Route B has cost 5 minutes so far and an estimated remaining time of 9 minutes.

f(B) = 5 + 9 = 14 minutes

A* would prefer route A because its estimated total cost is lower, assuming both routes remain feasible.

Unlike GBFS, A* does not choose only the location that seems closest. It considers the cost of getting there as well. This makes it more suitable when weather and terrain produce different movement costs. If the heuristic never overestimates the true remaining cost and the search conditions are appropriate, A* can find an optimal route.

For practical drone routing, the cost function should include safety. A risky route should receive a high penalty or be removed from the available choices. The heuristic should estimate remaining travel time or distance consistently with the cost function.

A* can require more memory and computation than GBFS because it stores and compares many candidate paths. In an emergency, it should use efficient data structures, a reasonable heuristic, and a time limit so that route planning remains responsive.

1.3 Analyze the impact of heuristic accuracy on both algorithms

A heuristic is an estimate of how far or how costly a state is from the goal. A straight-line distance is often useful because it is easy to calculate, but it may be inaccurate when the drone must avoid mountains, flooded areas, restricted zones, or strong winds.

For GBFS, the heuristic has a direct influence on the next choice. If the estimate makes a dangerous route appear close, GBFS may follow it even when another route is faster or safer. A misleading heuristic can therefore send the drone toward a dead end or force it to backtrack after receiving new information.

For A*, heuristic accuracy also affects efficiency. A more informative admissible heuristic can reduce the number of nodes expanded while preserving optimality. If h(n) is zero for every node, A* behaves like Uniform-Cost

Search and can still find an optimal route with non-negative costs, but it may explore many nodes. If h(n) overestimates the remaining cost, A* may become faster in some cases but loses its general optimality guarantee.

A useful drone heuristic could estimate straight-line travel time using a conservative speed estimate and then account for known wind direction or restricted zones. The estimate should remain consistent with the units and costs used by the route planner. Safety information should not be treated as a distance estimate alone; unsafe regions should also be represented in the route cost or as forbidden states.

1.4 Discuss how dynamic changes, such as blocked paths, affect search performance

A dynamic environment means that the map and travel costs can change while the drone is moving. A path that was open when the route was planned may become unsafe because of new weather data, obstacles, or a restricted landing area. A search result based on old information can no longer be trusted automatically.

When a path becomes blocked, the system must update the map, remove the blocked edge or assign it an effectively infinite cost, and calculate an alternative route. A basic A* search may need to restart or replan from the drone's current position. Repeated full searches can use time and battery, especially across a large region.

The system can improve performance by replanning only when meaningful changes occur, keeping current map information, and using incremental search methods such as Lifelong Planning A* or D* Lite. These methods reuse parts of earlier search work instead of solving the entire problem from the beginning.

The drone should also check the route during flight rather than treating the original plan as permanent. If sensor readings conflict with the map, it should slow down, choose a verified safe alternative, or enter a safe holding state where possible. In emergency delivery, a slightly longer route is preferable to continuing through a newly identified hazard.

1.5 Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

A* is the better starting choice when route quality, travel cost, and safety must be considered together. It accounts for the cost already travelled and the estimated remaining cost, whereas GBFS mainly follows the heuristic. This is important when a route that appears short can be slow or risky because of weather or terrain.

For a changing flood environment, I recommend A* combined with real-time map updates and an incremental replanning method such as D* Lite. The planner should assign high penalties to hazardous areas, exclude confirmed blocked paths, and update its route when new information arrives. A practical implementation can impose a computation deadline and use the best verified route found so far.

GBFS may be useful as a quick initial guide when computation time is extremely limited, but it should not be the only decision method for safety-critical delivery. The final system should prioritize safe, feasible routes first, then optimize delivery time. This balances responsiveness, route quality, and protection of the drone and the medicine.

2. Smart City Traffic Signal Optimization

A smart city needs to adjust traffic signals at many intersections to reduce total waiting time, fuel consumption, and congestion. The number of possible signal combinations is very large, and traffic conditions change continuously. The problem is therefore a large-scale optimization problem rather than a simple fixed-route search.

2.1 Formulate this as an optimization problem and explain why traditional search is inefficient

A possible solution is a complete set of signal timings for all intersections during a planning period. The variables may include green-light duration, phase order, and coordination offsets between nearby signals. The objective is to minimize a combined cost such as:

$J = \alpha W + \beta F + \gamma C$

Where:

- W: Total vehicle waiting time.
- F: Fuel consumption.
- C: A congestion measure.
- α, β, γ: Weights representing the importance of each objective.

For example, a candidate timing plan might produce W = 900 vehicle-minutes, F = 120 fuel units, and C = 70 congestion units. If $\alpha = 1$, $\beta = 2$, and $\gamma = 3$, its weighted cost is:

$J = 900 + 2(120) + 3(70)$

$J = 900 + 240 + 210$

$J = 1{,}350$

Another candidate can be compared using the same formula. The weights and units should be normalized in a real system so that one measure does not dominate only because of its scale.

Constraints include minimum and maximum green times, pedestrian crossing requirements, safe clearance intervals, and rules that prevent conflicting traffic movements from receiving green at the same time. The objective is to find a feasible timing plan with a low cost.

Traditional exhaustive search is inefficient because the number of combinations grows rapidly as intersections, phases, and timing choices increase. If 100 intersections each had 10 possible timing plans, the combined search space would contain 10^{100} configurations. Evaluating every option is not realistic in real time. Traffic also changes while the system searches, so a solution may be outdated before it is applied.

2.2 Explain how Hill Climbing would work in this scenario and identify its limitations

Hill Climbing begins with an initial traffic-signal plan and repeatedly examines nearby alternatives. A neighbour might increase one green phase by five seconds, reduce another phase, or adjust the offset between two connected intersections. The system evaluates each candidate using the objective function and moves to a neighbour with a lower cost.

For example, suppose the current plan has a cost J = 1,350. Changing a busy road's green duration reduces the cost to 1,280, so the system accepts the change. It then tests further small changes and continues while it can find an improvement. This is simple and can produce a useful result quickly.

Its main limitation is that it normally makes local improvements and may stop when no immediately neighbouring plan is better. The stopped plan may not be the best plan across the whole city. A change that temporarily increases waiting time at one junction might be needed to coordinate several junctions and reduce total congestion, but basic Hill Climbing may reject that move.

Hill Climbing can also be sensitive to its starting plan and the definition of neighbours. A poor starting plan can lead to a poor result. It is most useful for fast local adjustments, but a larger optimization method is needed when the city-wide solution has many interacting variables.

2.3 Discuss local maxima, plateaus, and ridges with real-world interpretation

A local optimum is a solution that is better than nearby alternatives but not necessarily the best overall solution. In a minimization problem, this is often called a local minimum. For traffic control, a signal plan may improve one junction but still leave the city with high overall waiting time because the surrounding intersections are poorly coordinated.

A plateau is a flat region where many nearby plans have almost the same objective value. For example, changing a green phase from 30 to 31 seconds may make no measurable difference because the traffic demand is low or the simulation rounds its results. Hill Climbing may stop or wander because it cannot identify a clearly better neighbour.

A ridge is a narrow path of improvement where progress requires coordinated changes to more than one variable. For example, increasing the green time at one junction and shifting the next junction's offset together may improve the flow of a traffic corridor. Changing only one timing at a time might not show the benefit, so the algorithm may miss the useful direction.

These problems show why a purely greedy local search can fail in a complex city. Multiple starting plans, random restarts, larger neighbourhoods, or methods that sometimes accept worse temporary solutions can improve the chance of finding better traffic timings.

2.4 Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

Simulated Annealing (SA) begins with a candidate plan and tests a neighbouring plan. It always accepts an improvement, but it may also accept a worse plan with a probability that depends on the size of the deterioration and a temperature parameter. At a high temperature, it explores more freely; as the temperature decreases, it becomes more selective.

For a minimization problem, a worse candidate with cost increase ΔJ can be accepted with probability:

$P = e^{(-\Delta J/T)}$

Where ΔJ is the increase in cost and T is the current temperature.

If $\Delta J = 10$ and $T = 20$:

$P = e^{(-10/20)}$

$P = e^{(-0.5)}$

$P \approx 0.607$

Later, if $T = 2$:

$P = e^{(-10/2)}$

$P = e^{(-5)}$

$P \approx 0.007$

This means the algorithm is more willing to explore early and less willing to accept worse plans near the end. This behaviour can help it escape local minima and cross plateaus.

A Genetic Algorithm (GA) maintains a population of candidate signal plans. Each candidate is encoded as a chromosome containing timings and offsets. The algorithm evaluates fitness, selects stronger candidates, combines parts of their plans through crossover, and introduces small random changes through mutation. Across generations, useful timing patterns can spread through the population.

SA is relatively simple and memory-efficient, while GA explores several candidate solutions at once and can handle many interacting variables. Both require careful design of the objective function and constraints. Infeasible signal plans must be repaired or penalized, and the number of simulations must be controlled to meet real-time limits.

2.5 Recommend the best approach and justify your choice based on scalability and adaptability

For city-wide optimization, a Genetic Algorithm is a strong choice when many intersections interact and the solution contains many variables. Its population-based search can explore different regions of the solution space instead of following one local path. It can also use past high-quality plans as starting candidates for the next optimization period.

However, running a large GA from scratch after every small traffic change may be too expensive. A practical system can use a GA periodically to optimize corridor-wide or city-wide coordination, then use fast local adjustments at individual intersections between major optimization runs. Simulated Annealing is also a good alternative when the solution encoding is simpler and memory or implementation complexity must be limited.

The recommended design is a hybrid approach: use a GA for broader coordination and a fast local controller for immediate sensor-based adjustments. Re-evaluate the objective using current traffic data, enforce pedestrian and road-safety constraints, and apply a new timing plan only when it is safe to do so. This combines scalability with responsiveness and avoids relying on a single local-search result.

3. Online Search for an Autonomous Mars Rover

An autonomous rover must explore unknown Martian terrain while collecting samples and avoiding hazards. Its sensors reveal only part of the environment, and communication delays make it impossible to depend on a human operator for every decision. The rover therefore needs to plan, act, observe, and revise its knowledge as exploration continues.

3.1 Explain why this problem requires an online search agent instead of offline search

Offline search assumes that the agent has a sufficiently complete model of the environment before planning. It can calculate a route using a known map and then follow that route. On Mars, the rover does not initially know the exact position of every rock, crater, slope, or soft-soil region, so an offline plan based on an incomplete map can be unsafe.

An online search agent makes decisions while interacting with the real environment. It chooses an action, observes the result, updates its internal map, and plans the next action. For example, the rover may initially select a route toward a sample location. After moving forward, its camera may reveal a crater blocking the path. The rover updates its map and selects a safe detour.

Online search is appropriate because the rover learns about the environment through movement and sensing. It does not need a complete map to begin operating, but it must keep checking whether its assumptions match the observations. It can also consider energy, communication limits, terrain difficulty, and the scientific value of possible sample sites.

3.2 Analyze the challenges of partial observability and dynamic environments

Partial observability means that the rover cannot see or measure every relevant part of the environment at once. A rock may hide a hole, a camera may not reveal soil strength, or a distant slope may be difficult to estimate. Sensor noise can also cause the rover to misjudge the location or size of an obstacle.

The rover should therefore maintain uncertainty in its internal model rather than treating every observation as perfectly accurate. It may mark a region as known safe, known hazardous, or not yet explored. When confidence is low, it should move cautiously, collect another observation, or choose a route with a larger safety margin.

The environment can also change from the rover's perspective as it moves: a newly visible obstacle changes the usable route, and dust or lighting conditions may reduce sensor quality. Although the terrain itself may not move, the rover's knowledge of it changes continuously. Mechanical condition, battery level, and wheel traction can also alter the cost of an action.

These challenges make it important to combine route planning with sensing, uncertainty management, and safety checks. The rover should avoid narrow routes close to hazards when the map is uncertain and should stop or replan when observations indicate a risk.

3.3 Describe how the rover builds and updates its internal model

The rover begins with a basic map, mission goals, and information from orbital images or earlier observations. It represents the terrain using a grid map, a topological map, or a three-dimensional map. In a grid map, each cell can store whether the area is free, occupied by an obstacle, or unknown. The cell can also store estimates of slope, roughness, and traversability.

As the rover receives camera, depth-sensor, and other instrument readings, it converts observations into map updates. If a sensor detects a rock in a previously unknown cell, that cell is marked occupied. If the rover safely traverses an area, the system can update its estimate of the terrain's traversability. Sensor fusion can combine multiple observations to reduce uncertainty.

The rover then assigns costs to possible actions. A flat, well-observed path may have a low cost, while a steep slope or uncertain region receives a higher cost. A sample location may provide scientific value, but the rover must balance that value against travel time, energy use, and risk.

After each movement, the rover compares the predicted result with the actual observation. If the expected position or terrain differs from reality, it updates the model and replans. Keeping a record of visited areas and previously attempted actions helps reduce repeated exploration and supports more efficient navigation.

3.4 Compare online search with uninformed and informed search strategies

Uninformed search methods such as Breadth-First Search (BFS) and Depth-First Search (DFS) do not use a heuristic estimate of how close a state is to the goal. BFS explores states in increasing depth and can find a shortest

path in an unweighted graph, but it may use large amounts of memory. DFS uses less memory in many cases but can follow an unhelpful route deeply before finding a solution.

Informed methods such as Greedy Best-First Search and A* use heuristic information. Greedy Best-First Search expands the state that appears closest to the goal, which can be fast but may ignore the cost already incurred. A* uses:

$f(n) = g(n) + h(n)$

It combines the cost so far with an estimate of the remaining cost. With a suitable admissible heuristic and appropriate conditions, A* can find an optimal path in a known graph.

Online search describes how the agent operates when it must act and learn in an unknown environment; uninformed and informed search describe how it chooses among available states. These ideas are not mutually exclusive. An online rover can use an informed method such as A* on its currently known map, then update that map and replan when new terrain is observed.

The main difference is that a purely offline planner can assume the map is available in advance, while an online planner must cope with incomplete knowledge and observations gathered during action. For the rover, a search method should be evaluated not only by path length but also by memory, energy, risk, and how well it responds to new information.

3.5 Justify the most suitable approach considering uncertainty, adaptability, and safety

The most suitable approach is an online planning system that combines an updated terrain map, a heuristic path planner, and explicit safety checks. A* can find a cost-effective route through the known map, while incremental replanning can reuse earlier work when a new obstacle is discovered. A method such as D* Lite is suitable for repeated route changes because it is designed to update plans as edge costs change.

The cost function should include distance, energy use, slope, rough terrain, and uncertainty. Dangerous cells can be forbidden, while uncertain cells receive a penalty until additional observations confirm that they are safe. The rover should also use a short planning horizon when immediate terrain is uncertain and extend its plan as more of the environment becomes known.

No algorithm can guarantee safety if sensors fail or the environment model is wrong. The rover therefore needs a safety layer that checks wheel traction, slope, obstacle distance, and battery limits before executing an action. If the next movement cannot be verified as safe, it should stop, gather more data, or choose another route. This approach supports exploration while reducing unnecessary risk.

4. University Exam Timetabling as a Constraint Satisfaction Problem

A university must schedule examinations while avoiding student clashes, respecting room capacity, observing subject-order requirements, and accounting for faculty availability and special accommodations. These requirements can be represented as a Constraint Satisfaction Problem (CSP), where a solution assigns values to variables while satisfying all required constraints.

4.1 Clearly define variables, domains, and constraints with explanation

Variables represent decisions that the timetable must make. For each course or examination, a variable can represent its assigned time slot and room. For example, X_Maths may represent the time slot for the Mathematics examination, while R_Maths represents the room allocated to it. A combined variable can also represent a time-room pair.

Domains are the possible values that each variable may take. If the university has five examination slots, the domain of X_Maths may initially be:

Domain(X_Maths) =

The room domain may contain rooms that are available and large enough for the number of candidates. If a room is unavailable at a particular time, that time-room combination should be removed from the domain.

Constraints specify which assignments are allowed. A student-clash constraint requires two examinations taken by the same student to occur in different time slots. A room-capacity constraint requires the room to hold all students assigned to the exam. A room-availability constraint prevents two exams from using the same room at the same time. Faculty constraints require an examination to be scheduled only when the required staff are available.

Precedence constraints represent requirements such as scheduling one subject before another. If course A must occur before course B, then:

Time(A) < Time(B)

Accommodation constraints may require accessible rooms, extra time, suitable seating, or a smaller room group for specific students. Hard constraints must always be satisfied, while preferences such as avoiding late slots can be treated as soft constraints to optimize after feasibility is achieved.

A valid solution is a complete assignment of time slots and rooms that satisfies all hard constraints. If the university also wants to reduce student gaps or balance the daily workload, these can be added as optimization preferences rather than treated as absolute restrictions.

4.2 Explain how backtracking search works in this scenario

Backtracking search assigns values to variables one at a time. It begins by selecting an exam, choosing a time and room from its domain, and checking whether the choice conflicts with assignments already made. If the assignment satisfies all constraints so far, the search continues with another exam.

For example, suppose Mathematics is assigned to slot S1 in Hall A. If Physics shares students with Mathematics, Physics cannot be assigned to S1. The algorithm tries another available slot, such as S2. If all remaining choices for a later exam cause a conflict, the algorithm returns to an earlier assignment and tries a different value. This process is called backtracking.

Backtracking is complete for a finite CSP when it systematically explores the available choices, so it can find a solution if one exists. However, the number of combinations can become very large as the number of exams and slots grows. The order in which variables and values are selected strongly affects the amount of search.

Efficiency can be improved by selecting the most constrained exam first, such as the exam with the fewest legal time-room options. This is the Minimum Remaining Values (MRV) heuristic. A degree heuristic can break ties by selecting the exam that constrains the largest number of other unassigned exams. These strategies often expose difficult conflicts early.

4.3 Discuss how constraint propagation, such as forward checking, improves efficiency

Constraint propagation reduces domains as soon as an assignment is made, preventing the search from continuing with choices that have already become impossible. Forward checking is a simple form of propagation: after assigning one variable, the algorithm removes inconsistent values from the domains of related unassigned variables.

For example, if Mathematics is placed in slot S1, forward checking removes S1 from the domains of all examinations that share students with Mathematics. If Physics then has only S2 and S3 remaining, the search does not waste time trying S1 for Physics. If a variable's domain becomes empty, the current assignment cannot lead to a valid timetable, so the algorithm backtracks immediately.

More advanced propagation, such as maintaining arc consistency, checks whether each remaining value has support in the domains of related variables. This can detect conflicts earlier, although it requires additional processing. The university should choose a level of propagation that provides a good balance between the cost of checking constraints and the reduction in search.

Combining backtracking, MRV, degree heuristics, and forward checking can significantly reduce the search space. The system should also maintain a clear representation of which students take each course, which rooms are available, and which faculty members are required so that constraint checks are accurate.

4.4 Analyze real-world challenges and justify the best strategy for scalability

Real-world timetabling involves incomplete or late registration data, changes in student enrolment, room maintenance, faculty leave, accessibility needs, and last-minute examination changes. Some courses have very large enrolments, and some students take combinations of subjects that create many conflicts. The system must also ensure that no student is assigned to two exams at the same time.

A practical solution should separate hard constraints from soft preferences. Hard constraints include no student clashes, room capacity, room availability, and required accommodations. Soft preferences may include avoiding several exams on consecutive days, distributing exams fairly, and reducing long gaps between examinations. First find a feasible timetable, then improve its quality without breaking hard constraints.

For scalability, use CSP backtracking with MRV, degree heuristics, forward checking, and constraint propagation. For very large timetables, the system can divide the problem by campus, department, or groups of time slots where possible, while checking cross-group student conflicts. Local-search methods can then improve a feasible timetable by swapping slots or rooms, provided every hard constraint remains satisfied.

The system should support incremental repair. When a room becomes unavailable, it should identify the affected exams and reassign them rather than rebuild the entire timetable unnecessarily. A final validation pass should check every hard constraint before publishing the timetable. This combination gives a structured, explainable, and scalable approach.

5. Strategic Game AI for Real-Time Decision Making: Minimax and Alpha-Beta Pruning

A gaming company needs an AI opponent that makes strong decisions against human players while operating under strict time limits. A strategy game may contain many possible actions at each turn, creating a large game tree. Minimax provides a way to select moves against an opponent, while Alpha-Beta pruning avoids exploring branches that cannot affect the final decision.

5.1 Explain how the Minimax algorithm models this problem

Minimax models a two-player, turn-taking game in which one player tries to maximize the outcome and the opponent tries to minimize it. The AI is commonly called MAX and the opponent MIN. The game is represented as a tree: each node is a game state and each edge is a legal move. Terminal states receive utility values representing outcomes such as a win, loss, or draw.

At a MAX node, Minimax chooses the child with the highest value. At a MIN node, it chooses the child with the lowest value because the opponent is assumed to play to reduce the AI's result. The values are propagated upward from terminal states to determine the best move at the root.

For example, suppose MAX has two possible moves, A and B. Move A leads to a MIN node whose possible outcomes have values 6 and 2, so MIN chooses 2. Move B leads to a MIN node with outcomes 4 and 3, so MIN chooses 3.

For move A:

Min(6, 2) = 2

For move B:

Min(4, 3) = 3

MAX compares 2 and 3 and selects move B, because 3 is better than 2.

In a complex strategy game, searching all the way to terminal states may be impossible. The algorithm therefore often searches to a fixed depth and uses an evaluation function to estimate the value of non-terminal states. The assumption that the opponent responds optimally makes Minimax a useful basis for competitive decision-making.

5.2 Analyze the computational challenges of large game trees

If each game state has an average branching factor b and the search explores to depth d, the number of nodes can grow approximately as:

O(b^d)

If b = 10 and d = 6, the number of leaf-level paths may be approximately:

$10^6 = 1{,}000{,}000$

Many additional internal nodes must also be evaluated. Increasing depth by only a few levels can multiply the work dramatically.

Strategy games often have many legal actions, including movement, resource collection, construction, and combat. Evaluating each state can also be expensive because the AI may need to consider resources, terrain, unit positions, objectives, and future threats. A full-width search may exceed the time available for a real-time move.

The game can also change while the AI is calculating if the system must respond to live events or a strict frame budget. Searching too deeply can cause delayed responses, while searching too shallowly can miss tactical threats. Memory usage can grow when the algorithm stores the search tree and associated state information.

The system therefore needs pruning, move ordering, a good evaluation function, and a strict computation budget. It may also use iterative deepening, which searches depth 1, then depth 2, and continues until the time limit is nearly reached. This ensures that the AI has a usable move even if deeper search must stop.

5.3 Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

Alpha-Beta pruning improves Minimax by avoiding branches that cannot change the decision at the root. It maintains two bounds:

- Alpha (α): The best value MAX can guarantee so far.
- Beta (β): The best value MIN can guarantee so far.

When alpha becomes greater than or equal to beta at a node, the remaining children of that node can be pruned because they cannot affect the final choice.

Consider the earlier example. MAX first explores move A, and the MIN node returns:

Min(6, 2) = 2

MAX's current best value, alpha, becomes 2.

MAX then explores move B. Suppose MIN examines a child with value 1. Because MIN can already force the value of B to be at most 1, and MAX already has an alternative worth 2, MAX will never choose B. The remaining children under B do not need to be examined; that branch is pruned.

The result of Alpha-Beta pruning is the same as ordinary Minimax when applied correctly; it does not change the chosen move. It only reduces the number of nodes evaluated. With excellent move ordering, the effective search can approach:

O(b^(d/2))

This allows a deeper search within the same time budget. With poor move ordering, its performance may be closer to ordinary Minimax's O(b^d).

Move ordering is therefore important. The AI should examine promising moves first, such as moves that capture valuable units, defend a critical objective, or create an immediate threat. Iterative deepening can provide information about strong moves from shallower searches, improving ordering at greater depths. Alpha-Beta pruning is particularly useful when the game tree is large and time is limited.

5.4 Design an evaluation function and justify the factors considered

A non-terminal game state needs a numerical evaluation so the AI can compare positions when it cannot search to the end of the game. A simple evaluation function can be written as:

$E(s) = w_1M + w_2R + w_3P + w_4O + w_5T - w_6D$

Where:

- M: Material or unit strength.

- R: Resource advantage.
- P: Positional advantage.
- O: Objective control.
- T: Tactical threat advantage.
- D: Damage or vulnerability.
- w_1 to w_6: Weights representing the importance of each factor.

For example, suppose a state has normalized values M = 5, R = 3, P = 2, O = 4, T = 1, and D = 2. With weights $w_1 = 3$, $w_2 = 1$, $w_3 = 2$, $w_4 = 4$, $w_5 = 2$, and $w_6 = 2$, the evaluation is:

$$E(s) = 3(5) + 1(3) + 2(2) + 4(4) + 2(1) - 2(2)$$

$$E(s) = 15 + 3 + 4 + 16 + 2 - 4$$

$$E(s) = 36$$

The number is not a guaranteed outcome; it is a score used to compare states consistently.

Material or unit strength measures the value of the AI's remaining units compared with the opponent's. Resources measure the ability to produce units or carry out future actions. Positioning captures control of important terrain, defensive positions, and access routes. Objective control measures progress toward the game's actual victory conditions. Tactical threats include immediate attacks, opportunities to capture units, or danger to important assets.

The factors and weights must match the rules of the specific game. If victory depends mainly on controlling objectives, objective control should receive a high weight. If the AI overvalues resources, it may collect resources while ignoring a critical battle. The evaluation should also use large positive or negative values for proven wins or losses so that a terminal result is always preferred to a merely favourable non-terminal position.

5.5 Recommend a strategy for balancing optimality and real-time performance

The recommended strategy is Minimax with Alpha-Beta pruning, iterative deepening, and a game-specific evaluation function. Alpha-Beta pruning reduces unnecessary exploration, while iterative deepening ensures that the AI always has a move available before the deadline. Good move ordering improves pruning and allows the search to examine promising actions first.

The AI should use a fixed time budget or node budget for each decision. It can search to increasing depths and keep the best move from the deepest completed iteration. If time runs out, it returns that move rather than delaying the game. A transposition table can store previously evaluated positions so that repeated states do not need to be searched again.

For especially large games, selective techniques such as beam search or carefully controlled extensions may be used, but they can sacrifice guarantees that full Minimax with Alpha-Beta provides. The AI should test its

evaluation function against different playing styles and measure response time, win rate, and missed tactical threats.

This design does not guarantee a perfect move in every real-time situation because the search may stop before reaching terminal states. However, it offers a strong practical balance: Minimax provides adversarial reasoning, Alpha-Beta reduces computation without changing the Minimax result, and time-aware iterative deepening ensures that decisions are delivered promptly.

Conclusion

The five scenarios demonstrate how search and optimization methods should be selected according to the environment and the constraints of the task. A* with dynamic replanning is suitable for emergency drone routing because it balances route cost and estimated remaining cost. Genetic Algorithms or Simulated Annealing can help optimize city-wide traffic timings where local search may become trapped. An online rover planner must continuously update its map and replan as new information is discovered. CSP techniques provide a structured way to generate exam timetables while satisfying hard constraints. Finally, Minimax with Alpha-Beta pruning, good move ordering, and iterative deepening provides a practical way to make strong game decisions under real-time limits.